

Challenge Técnico: Senior Front-end Developer

El ejercicio consiste en crear el frontend de una versión acotada de una red social en Typescript y React, consumiendo una API pública que proveemos, hecha con [mockApi](#). En esta app, el backend trabaja con solo dos recursos/DTOs:

```
export interface Post {
  createdAt: string;
  name: string;
  avatar: string;
  id: string;
  content: string;
  title: string;
}

export interface Comment {
  createdAt: string;
  name: string;
  avatar: string;
  id: string;
  content: string;
  parentId: null | string;
}
```

Aclaraciones

- name y avatar corresponden al usuario que creó el Post o Comment
- parentId tiene dos posibles valores:
 - null cuando es en respuesta a un post
 - string cuando es en respuesta a otro comentario

Requerimientos

1. La app deberá consistir de al menos dos pantallas:
 - a. Una pantalla principal donde se listan los posts
 - b. Una pantalla de detalles de un post donde se ve el contenido completo y también todos los comentarios asociados a este
2. El diseño queda a libre definición. Las únicas restricciones son:
 - a. No se deben usar librerías de componentes con estilos ya hechos (como **shadcn**, **antd**, **material ui** o **bootstrap**). Pero sí se permiten librerías de css utilitarias como **sass**, **tailwind** o **styled components** y librerías de componentes headless como **radixui** o **react aria**.
 - b. Los comentarios de comentarios deben verse en forma escalonada, como un árbol. Se tiene que poder ver claramente que un comentario tiene X comentarios en respuesta y a su vez cada uno de estos puede tener respuestas. En pocas palabras, es la forma en cómo resuelve este caso **reddit**.
3. Deberíamos poder Crear, Editar y Borrar Posts y Comentarios.
4. El proyecto debería subirse a un repositorio en cualquier plataforma.

5. Que en el repositorio se encuentre el Dockerfile correspondiente para buildear la imagen que sirva la aplicación, utilizando Nginx.

Sugerencias

- Buenas prácticas
- Escalabilidad de la aplicación
- Responsive
- Una buena estrategia de pedido de información al backend
- Hostear la aplicación en gh-pages o cualquier otro hosting
- Agregar tests de lo que se considere necesario

Endpoints

Para mas detalles sobre como funcionan y que features tienen los servicios de mockapi, pueden ir a <https://github.com/mockapi-io/docs/wiki/Code-examples>

getPosts

method: GET
url: https://665de6d7e88051d60408c32d.mockapi.io/post
body: {}
response: Post[]

getSinglePost

method: GET
url: https://665de6d7e88051d60408c32d.mockapi.io/post/\${postId}
body: {}
response: Post

createPost

method: POST
url: https://665de6d7e88051d60408c32d.mockapi.io/post
body: Partial<Post>
response: Post

deletePost

method: DELETE
url: https://665de6d7e88051d60408c32d.mockapi.io/post/\${postId}
body: {}
response: Post

getComments

method: GET

url: [https://665de6d7e88051d60408c32d.mockapi.io/post/\\${postId}/comments](https://665de6d7e88051d60408c32d.mockapi.io/post/${postId}/comments)

body: {}

response: Comment[]

createComment

method: POST

url: [https://665de6d7e88051d60408c32d.mockapi.io/post/\\${postId}/comments](https://665de6d7e88051d60408c32d.mockapi.io/post/${postId}/comments)

body: Partial<Comment>

response: Comment

deleteComment

method: DELETE

url:

[https://665de6d7e88051d60408c32d.mockapi.io/post/\\${postId}/comments/\\${commentId}](https://665de6d7e88051d60408c32d.mockapi.io/post/${postId}/comments/${commentId})

body: {}

response: Comment